

Labo 02 — Images, couches et registries

Théorie — comment une image est nommée, de quoi elle est faite, et d'où elle vient.

Objectifs

- Décomposer un nom d'image complet et savoir ce que Docker complète implicitement.
- Comprendre pourquoi un **tag** est une étiquette mouvante et un **digest** une identité.
- Expliquer le modèle en **couches** et ce qu'il implique pour le disque et le réseau.
- Savoir inspecter une image sans la lancer.
- Situer Docker Hub, les registries privés et les images « officielles ».

1. Le nom complet d'une image

Vous écrivez `docker pull postgres:16-alpine`. Docker, lui, comprend :

```
docker.io / library / postgres : 16-alpine
└─registry┐ └─namespace┐ └─repo┐ └─tag┐
```

Partie	Rôle	Valeur par défaut
registry	Le serveur qui héberge l'image	<code>docker.io</code> (Docker Hub)
namespace	L'organisation ou l'utilisateur propriétaire	<code>library</code> (images officielles)
repository	Le nom de l'application	<i>obligatoire</i>
tag	La version	<code>latest</code>

Trois conséquences immédiates :

- `postgres` seul signifie `docker.io/library/postgres:latest`.
- Une image d'entreprise porte un nom complet et explicite : `registry.masociete.fr/equipe-paiement/api-facturation:1.4.2`. La présence d'un **point** ou d'un **port** dans la première partie est ce qui indique à Docker qu'il s'agit d'un registry et non d'un namespace.
- Les images du namespace `library` sont les **images officielles** : maintenues en partenariat avec Docker, auditées, correctement documentées. `postgres`, `nginx`, `node`, `eclipse-temurin` en font partie. Une image `bobdu59/postgres` n'a aucune de ces garanties.

Piège — `latest` ne veut pas dire « la dernière version ». C'est un tag **par défaut** comme un autre, que le publieur choisit (ou pas) de déplacer. Il peut pointer vers une version vieille de deux ans. En entreprise, `latest` est proscrit en production : il rend le déploiement non reproductible et empêche tout retour arrière fiable.

2. Tag mouvant, digest immuable

Un **tag** est un pointeur : `postgres:16` désigne aujourd'hui `16.10`, demain `16.11`. Rien ne bouge sur votre disque, mais un nouveau `docker pull` ramènera autre chose.

Un **digest** est le condensat SHA-256 du contenu de l'image : `postgres@sha256:9d0d1f1e...`. Il est **calculé à partir du contenu**, donc :

- deux images de même digest sont bit à bit identiques, où qu'elles soient ;
- une image ne peut pas changer sans que son digest change ;
- on peut donc épingler un déploiement de façon absolue.

```
docker image inspect --format '{{index .RepoDigests 0}}' postgres:16-alpine
docker pull postgres@sha256:9d0d1f1e... # parfaitement reproductible
```

Le compromis usuel en entreprise : on *pousse* un tag unique et immuable par build (`api:1.4.2` ou `api:2026.03.17-b318`), on ne réutilise jamais un tag déjà publié, et les outils de déploiement épinglent le digest.

3. Une image est un empilement de couches

Chaque instruction de construction qui modifie le système de fichiers produit une **couche** (*layer*) : un ensemble de fichiers ajoutés, modifiés ou supprimés par rapport à l'état précédent. L'image finale est la superposition de ces couches, plus un manifeste qui les liste et une configuration (commande par défaut, variables, utilisateur...).

	couche 4 : COPY app.jar	(60 Mo)
	couche 3 : le JRE	(180 Mo)
	couche 2 : paquets système	(30 Mo)
	couche 1 : base Debian slim	(75 Mo)

↑ lecture seule, partagées entre toutes les images qui les contiennent

Cette structure explique quatre comportements que vous constaterez sans cesse :

1. Le partage sur disque. Si vos douze microservices partent du même JRE, ces 180 Mo sont stockés **une seule fois**. La somme de la colonne `SIZE` de `docker images` est donc très supérieure à l'espace réellement occupé — `docker system df` donne le vrai chiffre.

2. Le transfert différentiel. Un `pull` ou un `push` ne transfère que les couches absentes. D'où les `Already exists` en cascade lors d'un pull. Redéployer une nouvelle version de votre API ne transfère souvent que la couche du JAR, quelques dizaines de Mo, et non l'image entière.

3. L'immuabilité, y compris des erreurs. Une couche ne peut pas être modifiée après coup. Si une couche ajoute un mot de passe dans un fichier et qu'une couche ultérieure le supprime, **le fichier est toujours dans l'image** : la couche suivante ne fait que le masquer. Quiconque a l'image peut le récupérer. C'est la raison pour laquelle un secret ne doit jamais entrer dans un build (labo 08).

4. Le cache de construction. Puisque les couches sont identifiées par leur contenu, Docker réutilise celles qu'il a déjà. C'est le moteur du cache de build, que le labo 04 exploitera systématiquement.

À retenir — Le partage des couches se fait à l'échelle de l'hôte ou du registry, pas de l'image : une couche identique dans deux images différentes n'est stockée qu'une fois.

4. Multi-architecture : une image, plusieurs binaires

Un tag comme `postgres:16-alpine` ne désigne pas un seul contenu mais une **liste de manifestes** : une entrée pour `linux/amd64`, une pour `linux/arm64`, etc. Au `pull`, Docker choisit l'entrée qui correspond à votre machine.

C'est transparent jusqu'au jour où ça ne l'est plus : une image construite sur un MacBook Apple Silicon est `arm64` et refusera de démarrer sur un serveur `amd64`, avec un message `exec format error`. L'option `--platform` permet de forcer l'architecture voulue.

5. Les commandes du quotidien

```
docker pull nginx:alpine           # télécharger sans lancer
docker images                      # lister les images locales
docker image ls --filter dangling=true # les images sans tag (couches orphelines)
docker history nginx:alpine        # les couches, leur taille et leur origine
docker image inspect nginx:alpine  # métadonnées complètes en JSON
docker tag nginx:alpine mon-nginx:v1 # ajouter un nom à une image existante
docker rmi mon-nginx:v1            # supprimer un nom (et l'image si c'est le dernier)
docker system df                   # espace réellement occupé, avec le récupérable
```

Deux subtilités très mal comprises :

docker tag ne copie rien. Il ajoute une étiquette sur la même image ; les deux noms désignent le même `IMAGE ID`. Symétriquement, `docker rmi` sur une image portant deux tags ne supprime que le tag : les données ne partent que quand le dernier nom disparaît.

Une image « dangling » (`<none>:<none>`) n'est pas un déchet mystérieux. C'est une image dont le tag a été déplacé vers une version plus récente : elle a perdu son nom mais occupe toujours le disque. C'est le résidu normal des reconstructions successives.

Sortir une image du daemon

```
docker save -o api.tar mon-api:1.0 # archive complète, couches + métadonnées
docker load -i api.tar              # réimporte l'image, tags compris
```

Utile quand la machine cible n'a pas accès au registry (site isolé, poste sans réseau). Ne pas confondre avec `docker export` / `import`, qui manipulent le système de fichiers **d'un conteneur** et perdent au passage les couches et toute la configuration (`CMD`, `ENV`, `EXPOSE` ...).

6. Les registries

Un registry est un service HTTP qui stocke des couches et des manifestes.

Type	Exemples	Usage
Public	Docker Hub, <code>ghcr.io</code> , <code>quay.io</code>	Images de base et logiciels du commerce
Privé managé	AWS ECR, Azure ACR, Google AR	Images maison, hébergées chez le cloud
Privé auto-hébergé	Harbor, Nexus, GitLab Registry	Images maison, contrôle total, scan de vulnérabilités

Le cycle en entreprise est toujours le même :

```
docker login registry.masociete.fr
docker tag api:1.4.2 registry.masociete.fr/paiement/api:1.4.2
docker push registry.masociete.fr/paiement/api:1.4.2
```

Trois points à connaître :

- **docker login stocke le jeton côté client**, dans `~/.docker/config.json`, souvent en clair (encodé en base64). Sur un serveur partagé ou un agent de CI, c'est une fuite potentielle : on préfère les identifiants éphémères injectés par la CI.
- **C'est le daemon qui télécharge**, pas votre shell : le proxy d'entreprise doit être configuré pour le service `dockerd`.
- **Docker Hub limite les téléchargements anonymes** (quota par IP). Sur une CI qui tire des images en boucle, cela se manifeste par des `toomanyrequests` : d'où l'usage d'un *pull-through cache* ou d'une copie interne des images de base.

7. En entreprise

Sur une stack Spring Boot + Angular :

- Les **images de base** (`eclipse-temurin`, `node`, `nginx`, `postgres`) sont recopiées dans le registry interne. Personne ne tire directement d'Internet en production : quota, disponibilité, et surtout contrôle de ce qui entre.
- La CI construit `registry.interne/monapp/api:<version>` et `registry.interne/monapp/web:<version>`, puis pousse. La version vient du tag Git ou du numéro de build.
- Un outil de **scan** (Trivy, Harbor, Grype) analyse chaque image poussée et bloque celles qui portent des vulnérabilités critiques — d'où l'intérêt d'images de base minimales.
- Le déploiement référence une version précise, jamais `latest`.

À retenir

- Un nom complet est `registry/namespace/repository:tag` ; les valeurs par défaut sont `docker.io`, `library` et `latest`.
- `latest` n'est pas « la plus récente » : c'est un tag par défaut, à bannir en production.
- Le **tag** peut bouger, le **digest** `sha256:...` identifie un contenu exact.
- Une image est un empilement de **couches en lecture seule**, partagées entre images et transférées de façon différentielle.
- Un fichier supprimé dans une couche ultérieure reste présent dans l'image : jamais de secret dans un build.
- `docker tag` ne duplique rien ; `docker rmi` retire d'abord un nom, pas des données.
- `save` / `load` transportent une image complète ; `export` / `import` aplatissent un conteneur et perdent sa configuration.

Vocabulaire

repository : la collection des versions d'une image. — **tag** : étiquette mouvante d'une version. — **digest** : empreinte SHA-256 immuable du contenu. — **manifest** : document décrivant les

couches et la configuration d'une image. — **manifest list** : index multi-architecture. — **dangling image** : image ayant perdu son tag. — **layer** : couche de système de fichiers. — **pull-through cache** : miroir local d'un registry public. — **image officielle** : image du namespace `library` sur Docker Hub.